



CNRS - INP - UT3 - UT1 - UT2J

Institut de Recherche en Informatique de Toulouse



Embedded systems – Energy

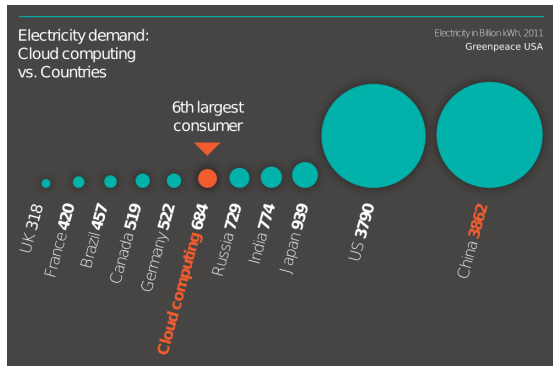
Georges Da Costa



neccampus

IT impact on electricity

- Recent datacenters: 40000 servers, 500000 services (virtual machines). Google, Facebook > 1 million servers
- One major power consumer
 - 2000 : 70 TWh
 - 2007 : 330 TWh, 2% of CO₂ world production
 - 2011 : 6th electricity consumer in the world
 - 2020 : 1000 TWh
- Rising
 - 2014 to 2016: 90% of datacenters were expected to need hardware upgrades





Sustainable datacenters

- Action can be done at several different levels
 - Hardware level: changing servers or cooling system
 - If entropy is constant, theoretical energy consumption is 0 !
 - Application level: rewrite applications while changing paradigm* or library
 - Middleware level: manages servers and services/applications
- Middleware: minimal cost, maximal impact
 - Clouds: OpenStack: 30% of market share in 2014
 - OpenSource solutions: 43% in 2014 (+72% in 2 years)
 - HPC: Slurm, LSF, plugin based

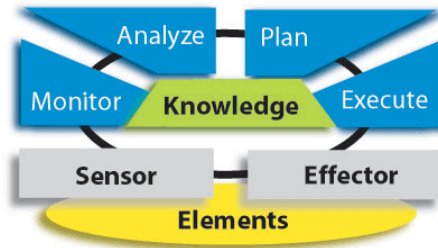


© ferloo.com

* Georges Da Costa et al. *Exascale machines require new programming paradigms and runtimes*, SFI journal, 2015

Middleware systems

- Two goals:
 - Managing (needs, errors, faults, overheating)
 - Optimizing (Energy, performance)
- Leverages
 - Switching on/off, DVFS
 - Scheduling, changing allocated resources
 - Migration (x86/ARM)*, suspend
- Monitoring
 - Hardware (performance counters, energy sensors)
 - High level (OS, application behavior)



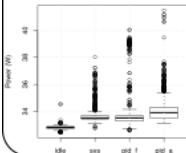
MAPE-K loop ©IBM

* Violaine Villebonnet et al., *Big, Medium, Little : Reaching Energy Proportionality with Heterogeneous Computing Scheduler*, Parallel

Complete loop

Complex sensors

Impact of measures



Measure

Analyse

Linear Programming

$$\begin{aligned} \forall j \quad \sum_k \alpha_{jk} = 1, \quad \forall j, h \quad \alpha_{jh} \in \{0, 1\}, \quad \alpha_{jh} \leq \alpha_h, \quad \alpha_{jh} \leq \alpha_j \\ \forall j, h \quad \alpha_{jh} \leq (1 - \alpha_h) + \alpha_j \\ \forall h \quad \sum_j \alpha_{jh} / r_j \leq \rho_h \\ \forall h \quad \sum_k \alpha_{hk} \leq \rho_h \end{aligned}$$

Réduire la puissance sous contrainte de performance :

$$\min(\sum_h (p_h^{old} + \sum_j \alpha_{jh} \times (p_h^{old} - p_h^{new})))$$

Metrics



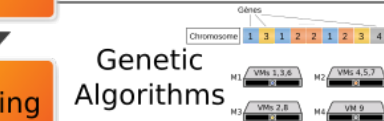
Real leverages



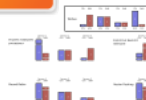
Execute

Planing

Genetic Algorithms



Simulations
MELBOURNE
CLOUDS
LAB



Vector packing



Outline: How to efficiently manage a datacenter

- Efficiently?
 - It is necessary to be able to compare (**models**)
- Managing means deciding
 - **Measure** tools
 - **Evaluation tools** : Experiments, simulation
 - Exact approaches and heuristics for **decision**
- **Node centric** point of view
 - Multi-levels optimization





Plan

- 1 Autonomic loop
- 2 Decision
- 3 Node optimization
- 4 Current projects and conclusion
- 5 Kernel-level



Model a system

To manage a system, we need to:

- Know all possible actions
- Know which is(are) the best one(s)

It can be translated into:

- Modeling impact and means (time, energy,...) of these actions
- Being able to compare two scenarios



Impact of leverages, an example with DVFS

Dynamic electric power consumed by a CMOS component:

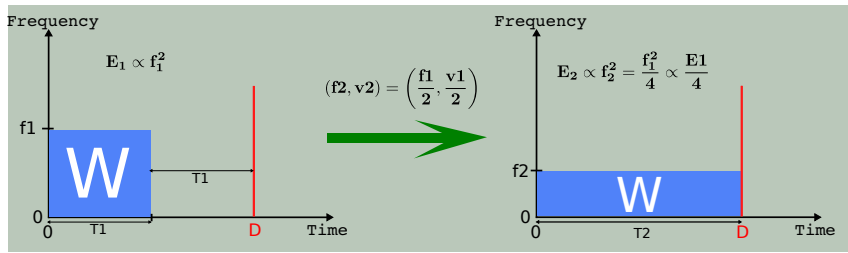
$$P_{cmos} = C_{eff} \times V^2 \times f$$

with, C_{eff} the effective capacitance *, V the voltage and f the frequency

* physical quantity: capacity of a component to resist to the change of voltage between its pins

Energy consumed for each tasks:

$$E = P * T \propto T * V^3, \text{ avec } V \propto f \text{ et } T \propto 1/F, \text{ alors } E \propto f^2$$





Even models are complex

Electrical power models for a single server:

- Classical : linear (error $E \sim 10-15\%$)

$$Power = P_{min} + Load \times (P_{max} - P_{min})$$



Even models are complex

Electrical power models for a single server:

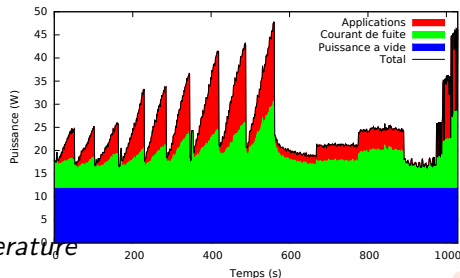
- Classical : linear (error $E \sim 10-15\%$)
- Finer : Processor voltage/frequency ($E \sim 5-9\%$)

$$Power = P_{min} + Load \times \alpha Voltage^2 Frequency$$

Even models are complex

Electrical power models for a single server:

- Classical : linear (error $E \sim 10-15\%$)
- Finer : Processor voltage/frequency ($E \sim 5-9\%$)
- Even finer: Processor temperature ($E \sim 4-7\%$)



$$Power = P_{min} + Load \times \alpha Voltage^2 Frequency + \lambda Temperature$$

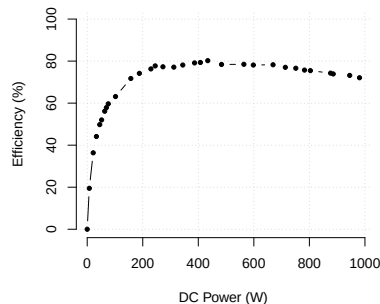


Even models are complex

Electrical power models for a single server:

- Classical : linear (error $E \sim 10-15\%$)
- Finer : Processor voltage/frequency ($E \sim 5-9\%$)
- Even finer: Processor temperature ($E \sim 4-7\%$)
- Do not forget about bias: **power supply unit** $E \sim 2-3\%$, cooling, ...

$$Power_{DC} = \omega_0 + \omega_1 Power_{AC} + \omega_2 Power_{AC}^3$$

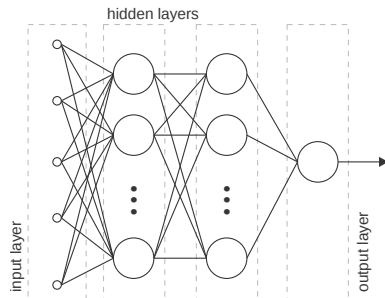


Even models are complex

Electrical power models for a single server:

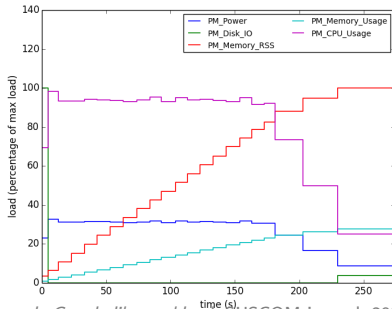
- Classical : linear (error $E \sim 10-15\%$)
- Finer : Processor voltage/frequency ($E \sim 5-9\%$)
- Even finer: Processor temperature ($E \sim 4-7\%$)
- Do not forget about bias: **power supply unit** $E \sim 2-3\%$, cooling, ...
- Learning methods (neural networks, $E \sim 2\%$) *

* Da Costa et al., *Effectiveness of neural networks for power modeling for Cloud and HPC: It's worth it!*, Transactions on Modeling and Performance Evaluation of Computing Systems journal, 2020



Modeling a datacenter is a complex task

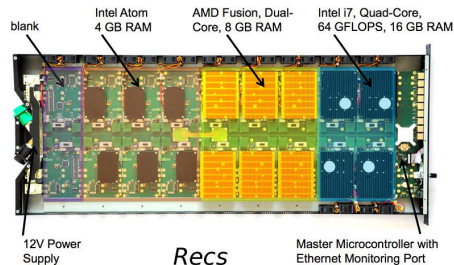
- Large number of elements
 - Applications
 - Process: Traces, high-level monitoring then abstraction*



* Georges Da Costa et al., *Modeling, classifying and generating large-scale Google-like workload*, SUSCOM journal, 2018.

Modeling a datacenter is a complex task

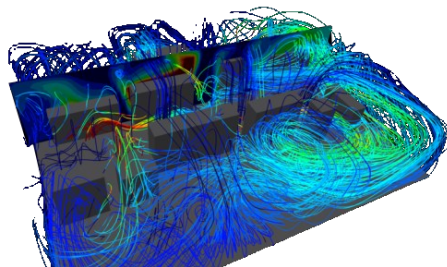
- Large number of elements
 - Applications
 - Process: Traces, high-level monitoring then abstraction*
 - Servers



* Georges Da Costa et al., *Modeling, classifying and generating large-scale Google-like workload*, SUSCOM journal, 2018.

Modeling a datacenter is a complex task

- Large number of elements
 - Applications
 - Process: Traces, high-level monitoring then abstraction*
 - Servers
 - Infrastructure

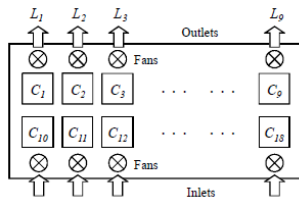


* Georges Da Costa et al., *Modeling, classifying and generating large-scale Google-like workload*, SUSCOM journal, 2018.



Modeling a datacenter is a complex task

- Large number of elements
 - Applications
 - Process: Traces, high-level monitoring then abstraction*
 - Servers
 - Infrastructure
- And their interactions
 - Thermal (D-Matrix)[†]
 - Between applications



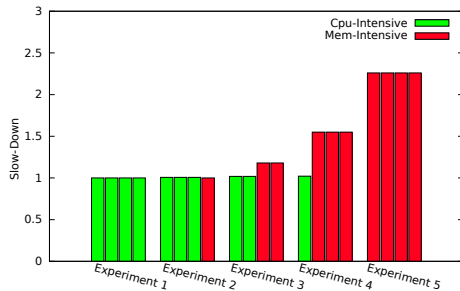
$$d_{x,k} = \begin{cases} 1, & \text{if } x = k \\ 0.84, & \text{if } x = k + 9 \\ 0, & \text{otherwise} \end{cases}$$

* Georges Da Costa et al., *Modeling, classifying and generating large-scale Google-like workload*, SUSCOM journal, 2018.

[†] Hong Yang et al., *Energy-efficient and thermal-aware resource management for heterogeneous datacenters*, SUSCOM journal, 2014.

A “simple” interaction of applications

- Two types of mono-thread applications
 - Application 1 : Cpu-Intensive, limited by the processor
 - Application 2 : Mem-Intensive, limited by memory
- Execution on a quad-core
 - Applications 1 : Independent
 - Applications 2 : Strong cross-impact





Metrics : A complex landscape

- HPC
 - Improve performance, throughput
 - Steady and known workload
- Cloud systems
 - Improve cost efficiency
 - Varying workload, difficult to predict
- Two main questions :
 - How to program*them at large scale?
 - How to manage them at runtime?

* Georges et al. *Exascale machines require new programming paradigms and runtimes*, SFI journal, 2015



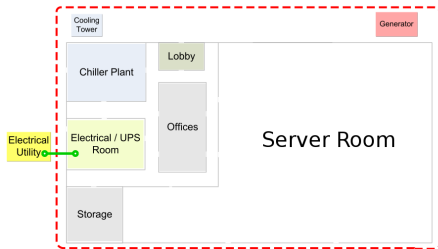
Metrics

- Direct values:
 - Processor and memory load, power, temperature,...
 - Objective: Does the system works? Comparing two datacenters, middleware, software,...
- 40000 servers, 500000 services → Need of simple metrics
 - Consumption **and** performance
- Difficult to standardize, mainly performance
 - Depends of the service, its implementation,...
- Classical metric: PUE

Georges et al. *Data Centres Sustainability Cluster Activities Task 3*. Rapport de recherche 3. European Commission, 2014

PUE : Power Usage Effectiveness

- Ratio Total electricity/IT electricity
- Mean value: 1.7 in 2014
- Standard initiated by GreenGrid
- Where does the IT part stops?
 - Power Supply Unit? Fans on the motherboard? Processor?
- Useful only in a very specific case



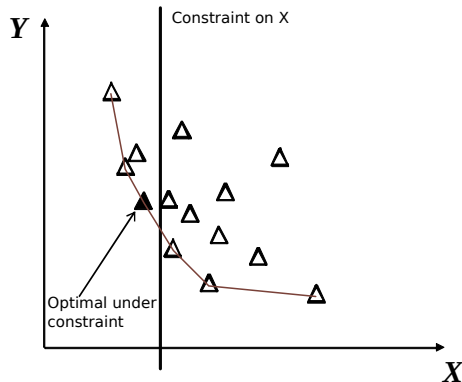


PUE : Power Usage Effectiveness

- Ratio Total electricity/IT electricity
- Mean value: 1.7 in 2014
- Standard initiated by GreenGrid
- Where does the IT part stops?
 - Power Supply Unit? Fans on the motherboard? Processor?
- Useful only in a very specific case
 - Constant overhead (100), IT part 100 to 200 depending of the load
 - For the same service provided by two softwares
 - 1 Mean load 75%
 $PUE = 275/175 = 1.57$
 - 2 Mean load 100%
 $PUE = 300/200 = 1.5$

Problem of multi-objective

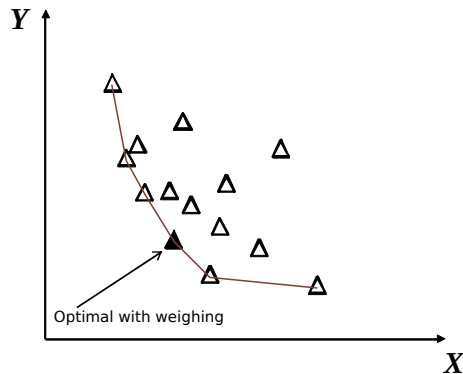
- Impossible to define in absolute, need a context, a goal
- Formalize simple metrics* : Dynamism, Energy, Performance, Resilience
- Several classical methods
 - Constraint optimization



* Tom et al., *Quality of Service Modeling for Green Scheduling in Clouds*, SUSCOM journal, 2014.

Problem of multi-objective

- Impossible to define in absolute, need a context, a goal
- Formalize simple metrics* : Dynamism, Energy, Performance, Resilience
- Several classical methods
 - Constraint optimization
 - Objective weighing

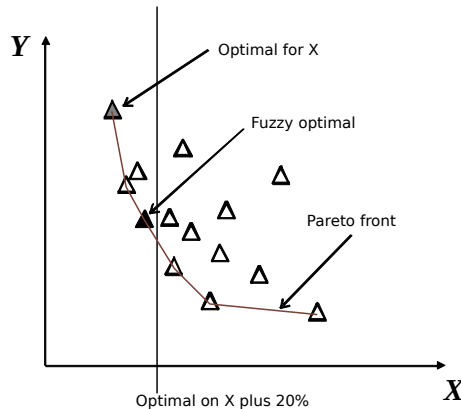


* Tom et al., *Quality of Service Modeling for Green Scheduling in Clouds*, SUSCOM journal, 2014.

Problem of multi-objective

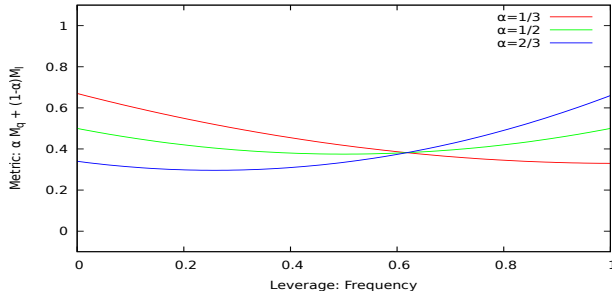
- Impossible to define in absolute, need a context, a goal
- Formalize simple metrics* : Dynamism, Energy, Performance, Resilience
- Several classical methods
 - Constraint optimization
 - Objective weighing
 - Fuzzy weighing[†] (Constraining by relaxation of optimal)

[†] Hong Yang et al. *Energy-efficient and thermal-aware resource management for heterogeneous datacenters*, SUSCOM journal, 2014.

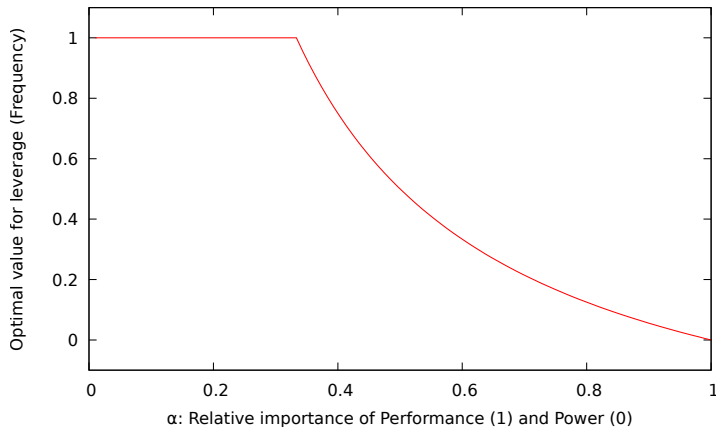


Example: Performance and Power

- Two metrics, one linear, one quadratic
 - M_q is the power, so it is quadratic in function of the frequency (x)
 - M_l is the performance, so it is linear in function of the frequency (x)
- $Obj_\alpha(x) = \alpha M_q + (1 - \alpha) M_l = \alpha x^2 + (1 - \alpha)(1 - x)$
- α is the weighing coefficient



How to choose α ?



Optimal value of frequency in function of α value.

- $Obj_{\alpha}(x) = \alpha M_q + (1 - \alpha) M_l = \alpha x^2 + (1 - \alpha)(1 - x)$
- $\alpha = 0$: Max frequency
- $\alpha = 1$: Min frequency
- In-between... Voodoo !



Test suite

- Two main categories:
 - Dedicated suites (Web services, database, HPC,...)
 - Generic suites
- Scientific, Infrastructure manager: Black-Box applications
- The system must be the same in all cases
- Maximum coverage test-suite
 - Same resources/Different power
 - Different resources/Same power

Georges et al., *Energy- and Heat-aware HPC Benchmarks*, EuroEcoDc workshop, 2013



Conclusion

Study of a system

- For which use?
- Three notions are linked to provide the answer:
 - Balance of the precision front of the models*
 - Objective function used for comparing
 - Scenarios used to actually compare

* Georges et al., *Modèles fluides pour l'économie d'énergie dans les grilles par migration : une première approche*, RenPar conference,

2009



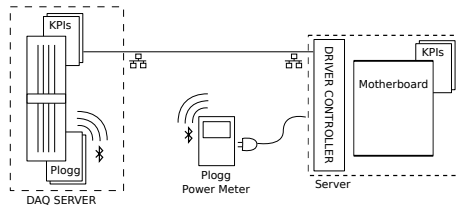
Measure Infrastructure

- Basis for taking decision
- Basis for metric evaluation
 - Classical Infrastructure (*nagios*, *ganglia*, ...)
 - Problem for scaling
 - Most values are unused of aggregated late
 - Some measures (processor, memory), but no **knowledge**
 - Need of higher level measures
 - What type of (phase of an) application
 - Electric power consumed by applications

Measuring Power Consumption

Complex problem

- Application and power measures are not on the same server
 - Synchronization of measurements

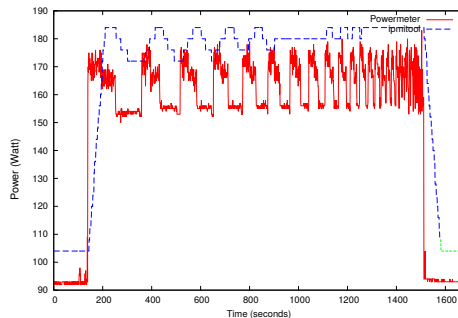


Da Costa et al., *Mastering system and power measures for servers in datacenter*, SUSCOM journal, 2017.

Measuring Power Consumption

Complex problem

- Application and power measures are not on the same server
 - Synchronization of measurements
- Internal or external power-meter ?
 - Usually external is more precise



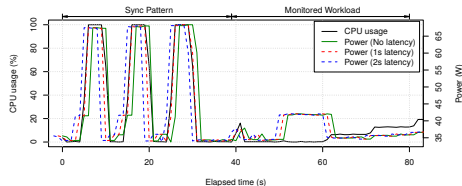
Da Costa et al., *Mastering system and power measures for servers in datacenter*, SUSCOM journal, 2017.



Measuring Power Consumption

Complex problem

- Application and power measures are not on the same server
 - Synchronization of measurements
- Internal or external power-metter ?
 - Usually external is more precise
- Different timing jitter between measure systems
 - Use starting and ending patterns

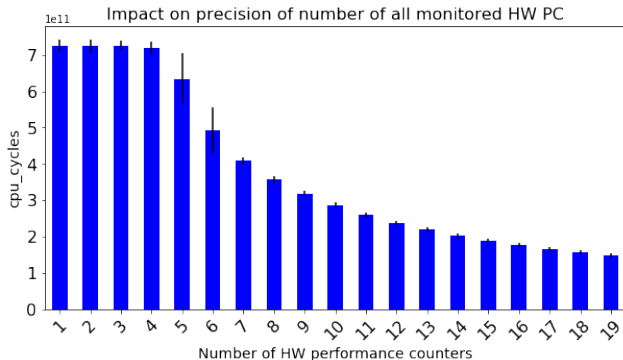


Da Costa et al., *Mastering system and power measures for servers in datacenter*, SUSCOM journal, 2017.



Monitoring tools

- Overhead is low
 - 0.2 to 2 milisec. for 1 to 20 Hardware performance counters
- Precision is critical
 - Machine learning depends on precision
- Dimension reduction
 - ACP, ...



<https://gitlab.irit.fr/sepia-pub/mojitos>

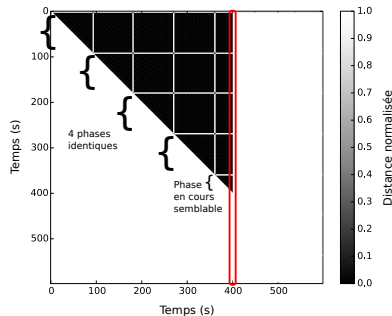
<https://hal.archives-ouvertes.fr/hal-03453537>



Which (phase of an) application is running

- A phase : behavior locally regular
 - Equivalent as a constant resource consumption
 - System measures constants
- Detection then identification
 - Signature of a phase
 - Same Phase $\sim$ Same Impact

Landry et al., *Exploiting performance counters to predict and improve energy performance of HPC systems*, FGCS journal, 2014.



Matrix of similar system measures (WRF : Weather
Research and Forecasting)



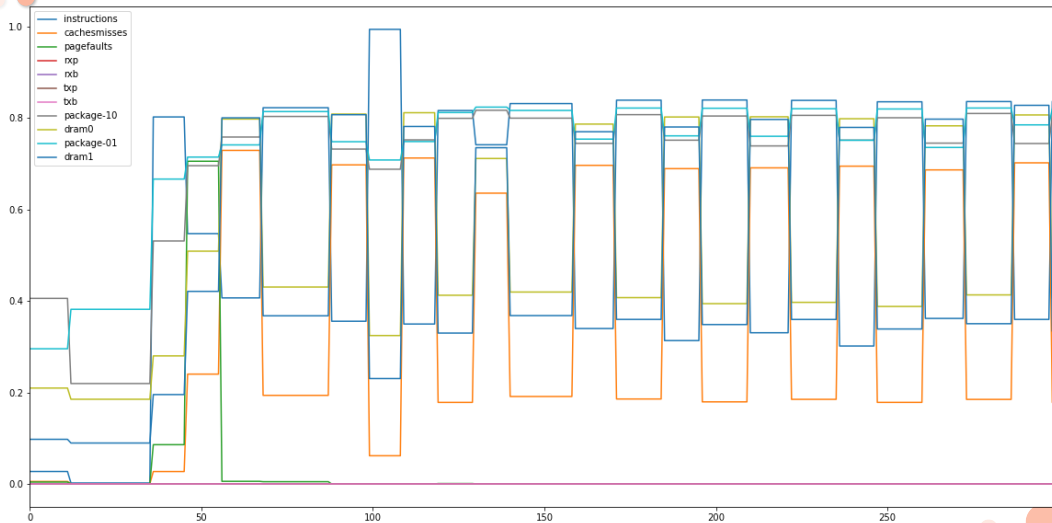
LU Diagonalization, raw values





LU Diagonalization, after phase detection

Autonomic loop Decision Node centric Conclusion Kernel-level
Models Measures Evaluation tools





External application identification

- Monitoring system values is intrusive
- Reduce the number of values monitored
- Using external values has lower impact (power, network)
- Authorize statistic tools
- Study the behavior during time

Georges et al., *Characterizing applications from power*

consumption : A case study for HPC benchmarks, ICT-GLOW

Symposium, 2011



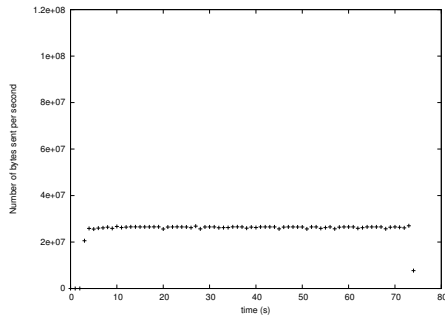
External application identification

- Monitoring system values is intrusive
- Reduce the number of values monitored
- Using external values has lower impact (power, network)
- Authorize statistic tools
- Study the behavior during time

Georges et al., *Characterizing applications from power*

consumption : A case study for HPC benchmarks, ICT-GLOW

Symposium, 2011



benchmark CG (NPB)



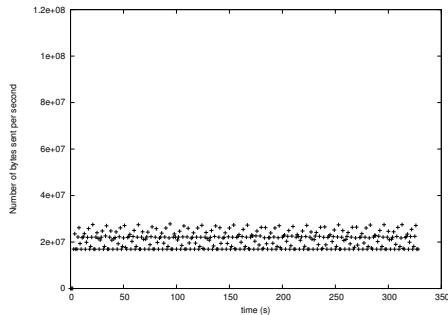
External application identification

- Monitoring system values is intrusive
- Reduce the number of values monitored
- Using external values has lower impact (power, network)
- Authorize statistic tools
- Study the behavior during time

Georges et al., *Characterizing applications from power*

consumption : A case study for HPC benchmarks, ICT-GLOW

Symposium, 2011



benchmark SP (NPB)



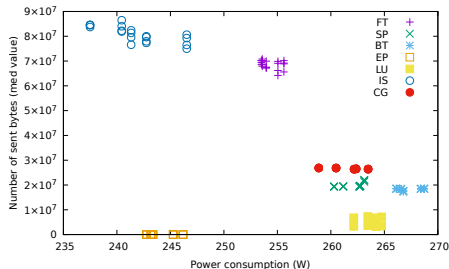
External application identification

- Monitoring system values is intrusive
- Reduce the number of values monitored
- Using external values has lower impact (power, network)
- Authorize statistic tools
- Study the behavior during time

Georges et al., *Characterizing applications from power*

consumption : A case study for HPC benchmarks, ICT-GLOW

Symposium, 2011



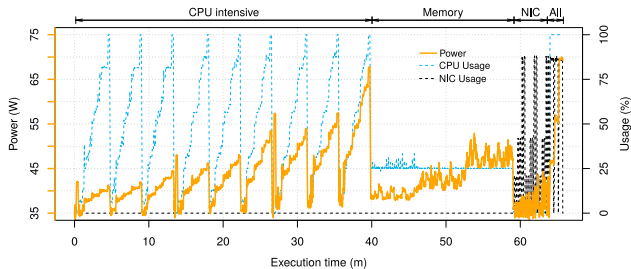


Power of servers and applications

- Watt-meters are not always available (application level: never)
- Model linking system measures with electrical power
- Analytical
 - Uses Datasheets. Very simple to put in place: PowerAPI

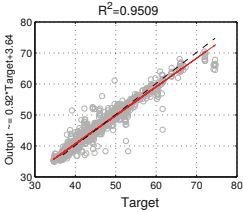
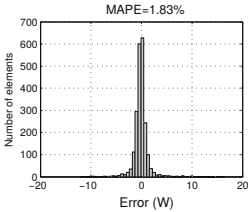
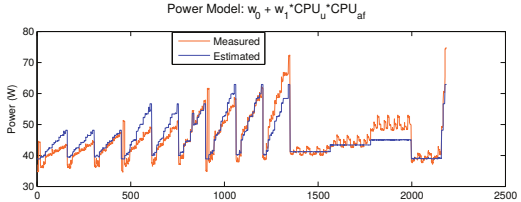
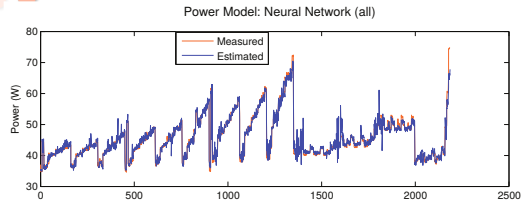
Power of servers and applications

- Watt-meters are not always available (application level: never)
- Model linking system measures with electrical power
- Learning method
 - Good coverage of the learning set and low impact of the measure

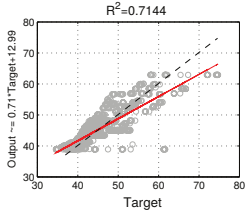
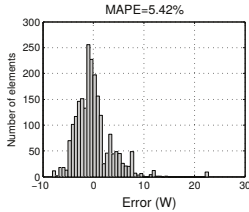




Power models: Neural network VS capacitive model



(a) ANN regression using all KPIs.

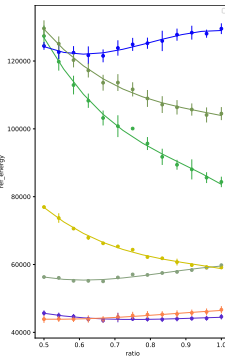
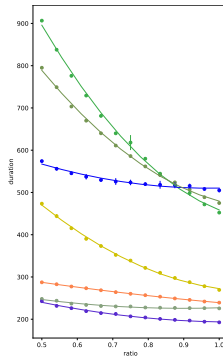
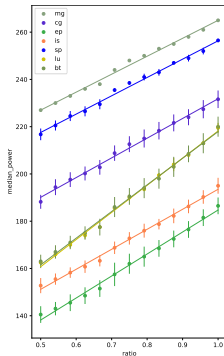


(b) Calibrated capacitive model.

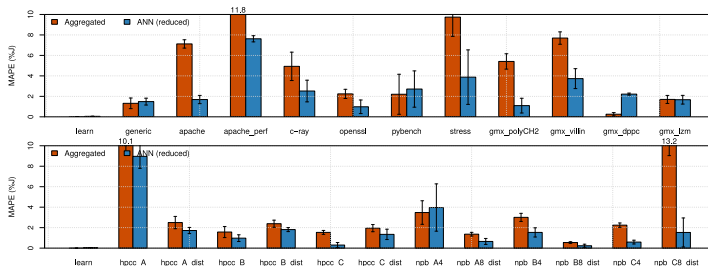


DVFS on modern servers is more complex

- Power is nearly linear
- Time is not linear
- Energy is ... something
- Depends completely on the application



Result of learning methods



Error of two models: **Aggregated** (linear regression on sum of sub-models) and **ANN** (neural network)

- Most models: error of 5%
 - Reference in the field: Rivoire et Al. *A Comparison of High-Level Full-System Power Models*

Leandro *Modeling the power consumption of computing systems and applications through Machine Learning techniques*, 2015



Conclusion

- Current approaches need a global point of view
 - Scalling
 - Latency problem
 - Lots of decisions are local (DVFS, migration,...)
- Open problems
 - Granularity of the measure: Adaptive and multi-scale
 - Spatial and Temporal independence
 - Maximal coverage test suite automatic generation

Almeida et al. *Energy Monitoring as an Essential Building Block Towards Sustainable Ultrascale Systems*, SUSCOM journal, 2017.



Evaluation tools: Experimentation, Simulation

- To improve, comparison is necessary
- Three main methods
 - Mathematical models
 - Simulation
 - Experiments



Linear programming

- Describe all constraints with linear equations

Example : A task is on a unique server

- Let e_{jh} the fact that task j runs on server h
- $e_{jh} = 1$ iif task j is on server h
- $\forall j, h \quad e_{jh} \in \{0, 1\},$
 $\forall j \quad \sum_h e_{jh} = 1$



Linear programming

- Describe all constraints with linear equations
- Describe the objective as a function to minimize

Example : Minimize the total power consumed

- P_h^{stat} et P_h^{dyn} : static and dynamic power of server h (linear model)
- Let α_{jh} the processor fraction of task j on server h
- $\min \sum_h (P_h^{stat} + \sum_j \alpha_{jh} P_h^{dyn})$



Linear programming

- Describe all constraints with linear equations
- Describe the objective as a function to minimize
- Formalize leverages and their impact
- Approximation of real world (quadratic phenomena)
- Exact resolution for small cases

$$\begin{aligned} \text{Constraints :} \\ \forall h \quad p_h \in \{0, 1\} \\ \forall j \quad \sum_h e_{jh} = 1, \quad \forall j, h \quad e_{jh} \in \{0, 1\}, \quad \alpha_{jh} \leq e_{jh}, \quad m_{jh} \leq e_{jh} \\ \forall j \quad v_j \in \{0, 1\}, \quad \forall h \quad \sum_j \alpha_{jh} \leq v_j, \quad \sum_j m_{jh} \leq v_j \\ \forall j, h \quad p_h \leq (1 - e_{jh}) + v_j \\ \forall j, h \quad v_j \leq (1 - e_{jh}) + p_h \\ \forall h \quad \sum_j \alpha_{jh} / r_j \leq v_j \\ \forall j \quad \sum_h m_{jh} \leq v_j \end{aligned}$$

Minimize power under performance constraints:

$$\begin{aligned} \forall j \quad \sum_h \alpha_{jh} / r_j > \text{Threshold} \\ \min(\sum_h (P_h^{\min} + \sum_j \alpha_{jh} \times (P_h^{\max} - P_h^{\min}))) \end{aligned}$$

Damien et al., *Energy-Aware Service Allocation*, FGCS journal, 2011



Scaling of linear programming

- Exact method: time complexity exponential in function of integer variables
 - 6 servers, 16 tasks : 3 minutes (GLPK)



Scaling of linear programming

- Exact method: time complexity exponential in function of integer variables
 - 6 servers, 16 tasks : 3 minutes (GLPK)
- Methods of constant variables
 - Fix some variables, solve, change the fixed variables and iterate
 - Worse than the optimal but can be very fast

Guerout et al., *Mixed integer linear programming for quality of service optimization in Clouds*, FGCS journal, 2017



Scaling of linear programming

- Exact method: time complexity exponential in function of integer variables
 - 6 servers, 16 tasks : 3 minutes (GLPK)
- Methods of constant variables
 - Fix some variables, solve, change the fixed variables and iterate
 - Worse than the optimal but can be very fast
- Relax the integer constraint
 - “Better” than optimal (a task half on a server and half on another one)

Guerout et al., *Mixed integer linear programming for quality of service optimization in Clouds*, FGCS journal, 2017



Scaling of linear programming

- Exact method: time complexity exponential in function of integer variables
 - 6 servers, 16 tasks : 3 minutes (GLPK)
- Methods of constant variables
 - Fix some variables, solve, change the fixed variables and iterate
 - Worse than the optimal but can be very fast
- Relax the integer constraint
 - “Better” than optimal (a task half on a server and half on another one)
- Using both it is possible to have a interval

Guerout et al., *Mixed integer linear programming for quality of service optimization in Clouds*, FGCS journal, 2017



Simulation

- Large number of simulators: SimGrid, DCWorms, CloudSim, ...
- Particular needs for our research
 - Cloud models (migration, Over-allocation of resources, federation[†])
 - DVFS
 - Electrical power
 - Temperature
- Situation is steadily improving
 - DVFS and fine-grained management of clouds in CloudSim
 - Thermal simulation in DCWorms*
 - DVFS and energy in SimGrid
- Now mainly BatSim

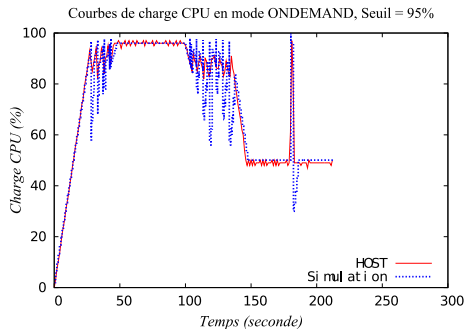
* Wojtek et al., *Energy and thermal models for simulation of workload and resource management in computing systems*, SMPT journal, 2015. [†]Thiam et al., *Cooperative Scheduling Anti-load balancing Algorithm for Cloud*, CCTS workshop, 2013

Adding DVFS in CloudSim

- Simulators mainly come from Grid world
 - Stability during time
 - Resources are always used at 100%
- DVFS needs to move events
- Fine-grained temporal management (1/10 s)

Tom et al., *Energy-aware simulation with DVFS*, SMPT journal,

2013





Ad-hoc simulators: Reinvent the wheel?

- Simplistic simulators
- Lets test an idea at a low cost
- Necessary to stop at the right complexity level
 - Simulator of heterogeneous architectures*
 - No network simulation
 - Example: prove the utility of heterogeneity to reach a proportional system

* Georges, *Heterogeneity: The Key to Achieve Power-Proportional Computing*, CCGrid conference, 2013.



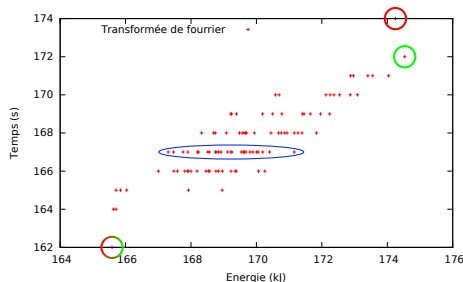
Experimentation

- A model is always an approximation
- Final validation by experiment
- Complex because of the need to have electrical measures
 - At ENS-Lyon, they were one of the first to experiment with watt-meters at large scale (GreenNet)*
- Problem of distributed measures, electrical conversions, impact of measures (performance counters)
- Reproducibility problem

* Georges et al., *The green-net framework: Energy efficiency in large scale distributed systems*, IPDPS, 2009

No stability of experiments

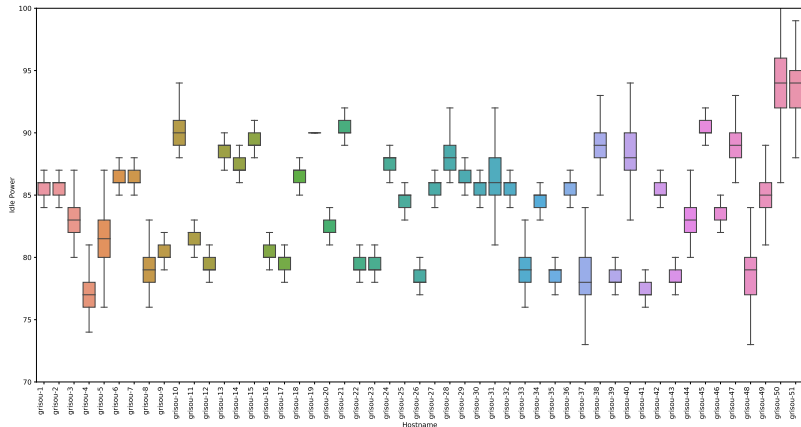
- Simple experiment of Fast Fourier Transform (NPB)
- 100 experiments on exactly the same hardware (Grid'5000)
- Large variations
 - Time: 12s, 7% (Std. Dev. 3.2s)
 - Energy: 9.3kJ, 5.5% (3kJ)
- For the same time, 167s
 - Difference of 4kJ
- Time $\neq$ Energy





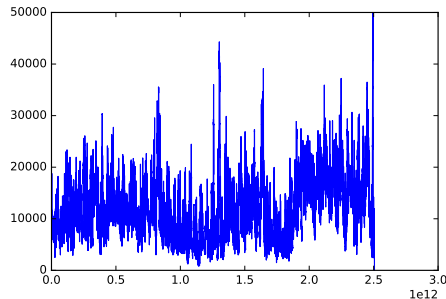
Heterogeneity between servers

- *Idle* servers
- Same hardware (Grid'5000)
- Same O/S: Debian 10.3, Linux kernel 4.19.0-8-amd64
Duration 500s
- Large variations
 - Min: 77.4W
 - Max: 93.8W



Workload Generation

- Large scale workload are difficult to find
- Google Datacenter: 21000 servers, 1 month
- Methodology
 - Extract distribution laws



Da Costa et al. *Modeling, Classifying and Generating large-scale Google-like Workload*, SUSCOM Journal 2018



Workload Generation

- Large scale workload are difficult to find
- Google Datacenter: 21000 servers, 1 month
- Methodology
 - Extract distribution laws
 - Generate similar laws

```
1 import scipy.stats, random, numpy
2
3 def truncated_expon(lamda):
4     while True:
5         val = scipy.stats.expon.rvs(scale=1.0/lamda)
6         if val <= 1.0:
7             return val
8
9 def get_makespan(mass, disparity):
10     mu = numpy.log(mass / disparity)
11     sigma = numpy.sqrt(2*(numpy.log(mass) - mu))
12     return scipy.stats.lognorm.rvs(sigma, scale = mass /
13                                     disparity)
14
15 def get_next_task(timestampLastEvent, dynamism, ratio, names,
16                   masses, disparities, priorities):
17     arrival = scipy.stats.pareto.rvs(4, loc=-1) * 3.0 * dynamism
18     newTimestamp = timestampLastEvent + arrival
19     typeIndex = random.choices(list(range(len(ratio))),
20                               weights=ratio)[0]
21     name = names[typeIndex]
22     priority = truncated_expon(priorities[typeIndex])
23     makespan = get_makespan(masses[typeIndex], disparities
24                             [typeIndex])
25     return (newTimestamp, name, priority, makespan)
```



Conclusion

- All tools are limited
 - Models: Approximate or optimal unreachable
 - Simulation: Approximate
 - Experimentation: Reproducibility and very sensible
- Large investment necessary
- Simulation : Quite good value for money
- Difficult to test In-Vivo



Plan

- 1 Autonomic loop
- 2 Decision
- 3 Node optimization
- 4 Current projects and conclusion
- 5 Kernel-level



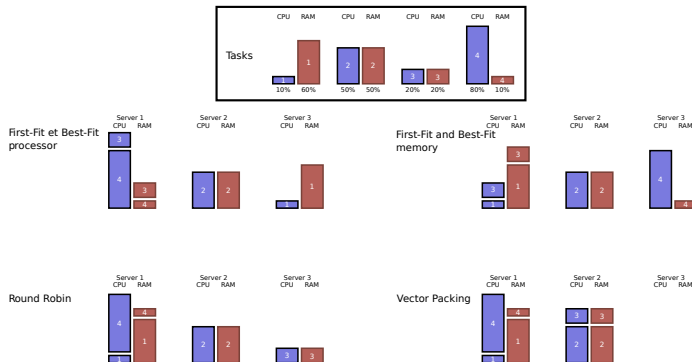
Exact Approaches and heuristics

- Two problems
 - Placement
 - Temporality
- Classical heuristics for placement
 - Greedy: Best Fit, First Fit
 - Vector Packing (*Gourmet Greedy*)
 - Genetic algorithms



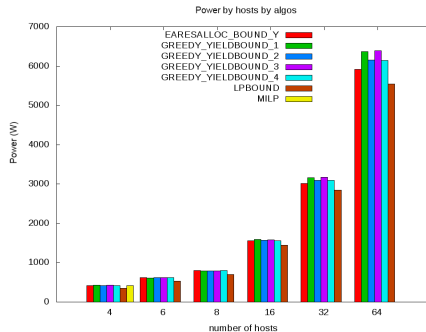
Classical greedy algorithms

- Characteristics
 - Memory
 - Processor
- Sort services
- Sort servers
- No coming back on previous decisions



Gourmet Vector packing

- 4 objectives in the sort function
 - Server is attractive from an energy point of view
 - Add the task do not overload the server
 - Server already switched on
 - The tasks brings back the balances of resources
- Time “only” in $\mathcal{O}(J \times H \ln(H))$
- But the solution of the *Gourmet* is difficult to qualify

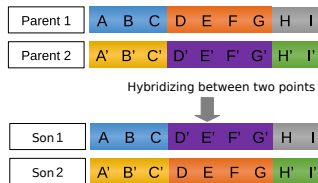
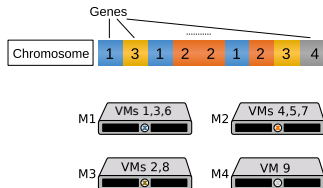


Damien et al., *Energy-Aware Service Allocation*, FGCS journal, 2012.

Genetic Algorithms

- Chromosome = Allocation
- Initial random generation
- At each generation:
 - Hybridizing and mutation
 - Sort on the objective metric
 - Keep only the best

Tom et al., *Quality of Service Modeling for Green Scheduling in Clouds*, SUSCOM journal, 2014





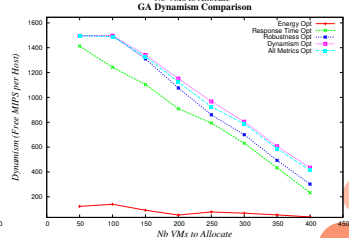
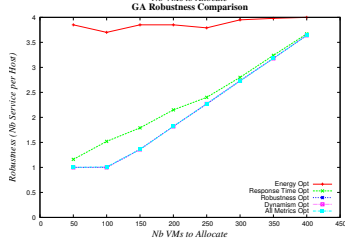
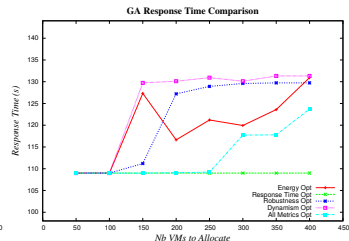
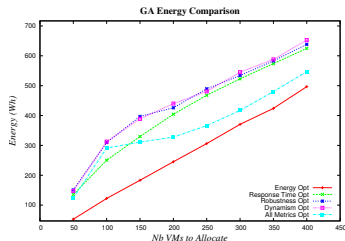
Metrics for genetic algorithms

- Contrary to with greedy, we can optimize a metric directly
- Examples of metrics
 - Energy, Performance, Resilience, Dynamism

GA Name	Coefficient applied to metrics			
	Energy	Response time	Robustness	Dynamism
GA All	1	1	1	1
GA Energy	1	0	0	0
GA RespT	0	1	0	0
GA Rob	0	0	1	0
GA Dyn	0	0	0	1

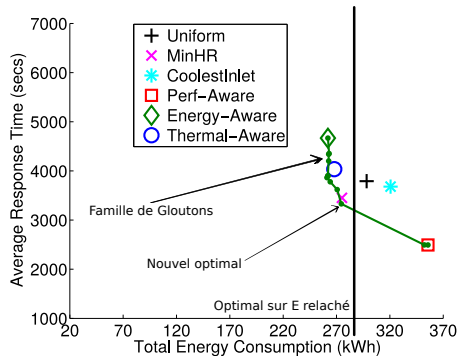
Results of the genetic algorithm

- Each algorithm is the best in its own domain (Energy)
- GA_All Very good everywhere
- 400 services on 110 servers, approximately 40s
- Taking into account a metric is already very important



Fuzzy Greedy

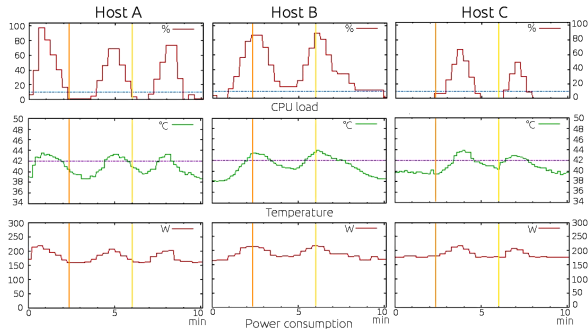
- Advantage of the G.A.: aim an objective
- Similar method for greedy algorithms
 - Families of greedy algorithms
 - Keep the best
 - Define the *best* ?
- Fuzzy multi-objective



Hong Yang et al., *Multi-Objective Scheduling for Heterogeneous Server Systems with Machine Placement*, CCGRID conference, 2014

Why power/energy is unique

- The temporal point of view
 - Inertia due to temperature
 - Switching on/off servers
 - Over- or Under-reservation
 - Cycles are sometime good
- Non-linearities
 - Equation of power
- Feedback loops
 - Cooling system



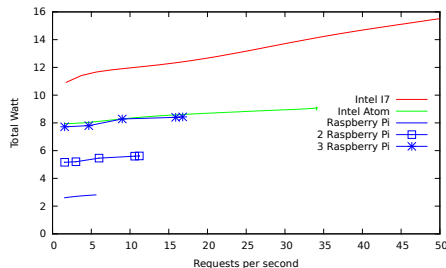
Violaine et al., *Thermal-aware cloud middleware to reduce cooling needs*,

WETICE workshop, 2014



Data center in the box

- Rack level
- Low number of services: High variance
- Perfect adaptation to the load
- Currently costly: Initial overhead
 - Cooling
 - Everything which is negative in the PUE
- *Proportional* architecture

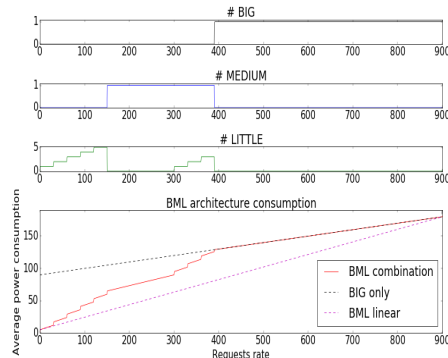


Violaïne et al., *Big, Medium, Little : Reaching Energy Proportionality with Heterogeneous Computing Scheduler*, Parallel Processing Letters, journal, 2015.



Data center in the box

- Rack level
- Low number of services: High variance
- Perfect adaptation to the load
- Currently costly: Initial overhead
 - Cooling
 - Everything which is negative in the PUE
- *Proportional* architecture



Violaïne et al., *Big, Medium, Little : Reaching Energy Proportionality with Heterogeneous Computing Scheduler*, Parallel Processing

Letters, journal, 2015.



Reaching energy-proportionality using heterogeneous hardware

Use nodes depending on the real load (web server as example), not the peak load

Example:

Processor	Watt range	Max request/s	Efficiency (W/r)
Intel I7	11 - 42	353	.12
Intel Atom	8 - 9	34	.26
Raspberry Pi	2.56 - 2.81	5.6	.50

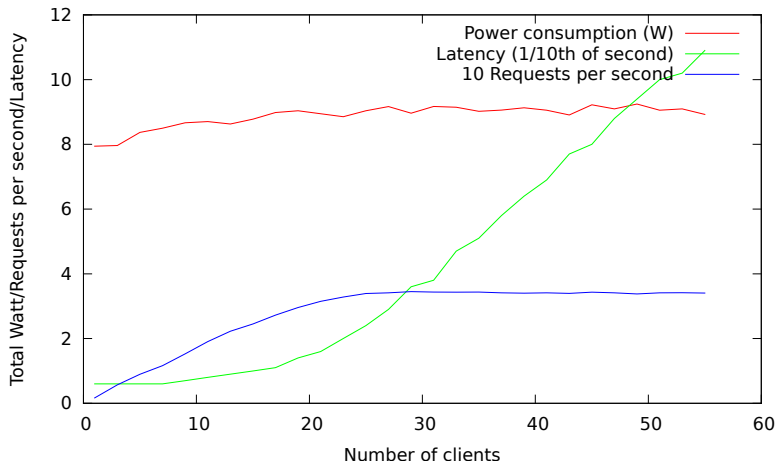
Intuition: Several small node and intermediary nodes to have a multi-scale smooth curve

Zoom on Intel I7

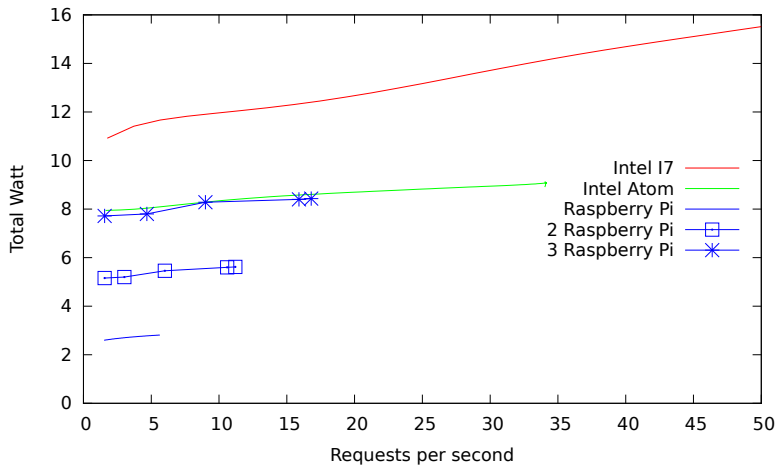




Zoom on Intel Atom



Comparison





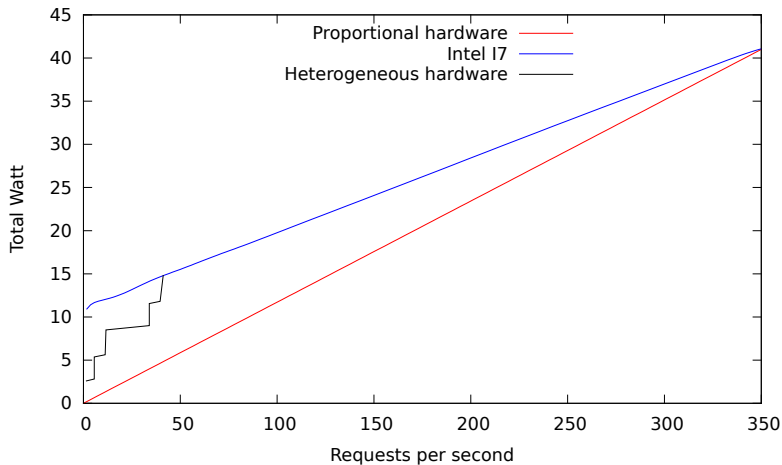
A near-linear behavior

Take the most efficient hardware as function of workload:

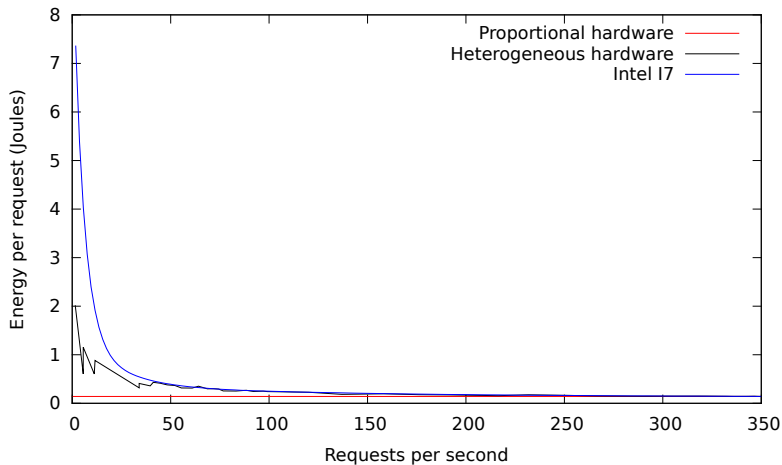
0→5 req/s	1 Raspberry Pi
5→10 req/s	2 Raspberry Pi
10→35 req/s	1 Intel Atom
35→40 req/s	1 Intel Atom + 1 Raspberry Pi
40→350 req/s	1 Intel I7

For over 350req/s, use modulo 350, and use as many I7 as necessary

Still far far away...



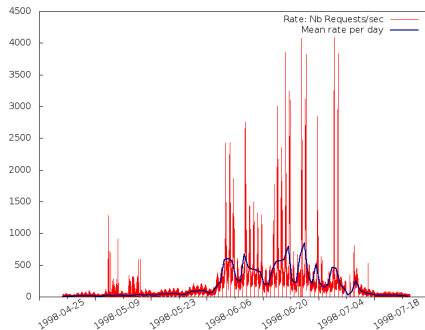
Not so far, efficiency view !

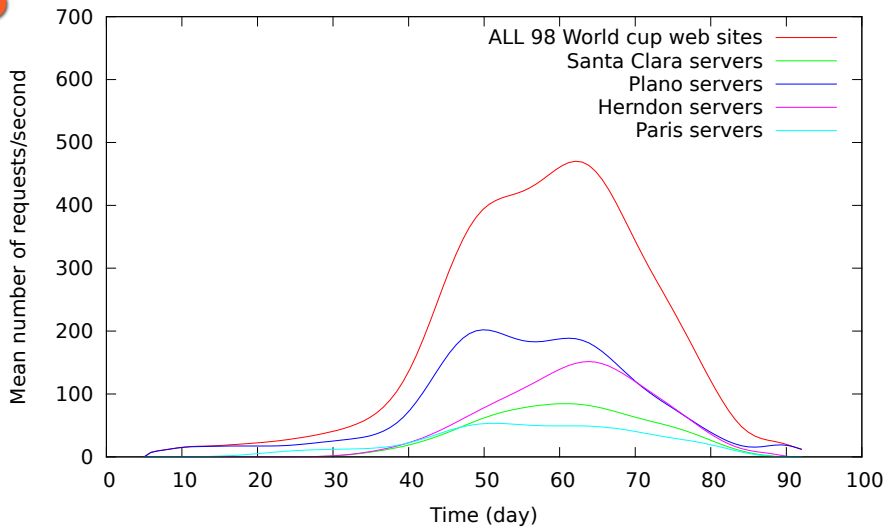




98 Football World Cup

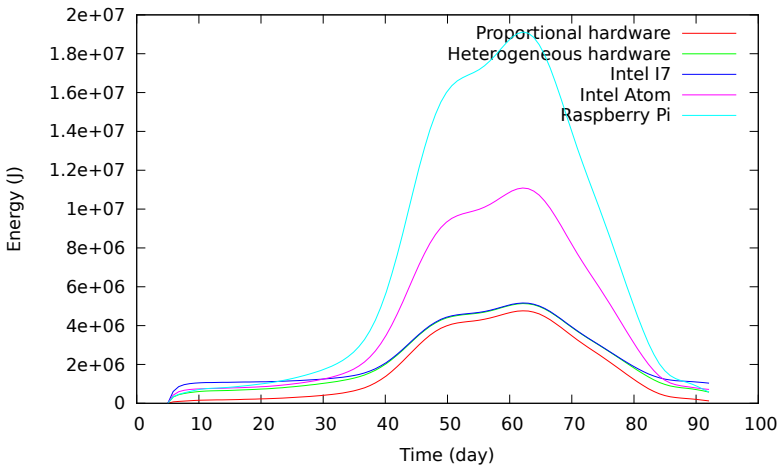
- Available data
- 92 days of web server access logs
- Workload precise at the second level
- Four geographic locations, three in US, one in France
- Several phases
 - Low phases, first 40 days and last 10 days
 - High phase, during the competition





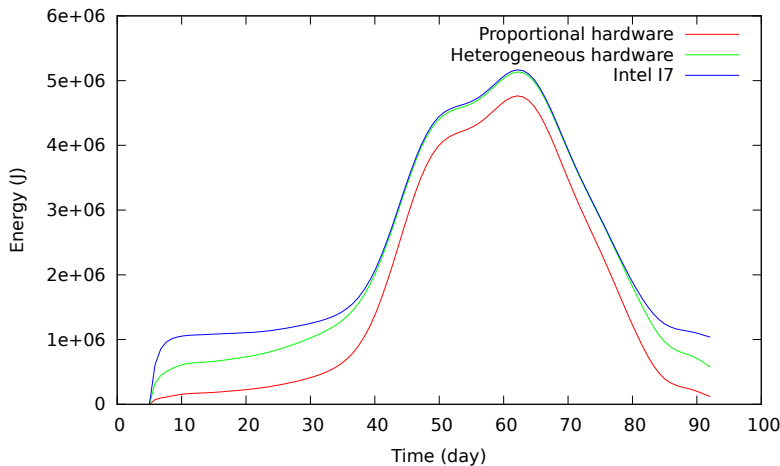


Comparison if using one single data-center

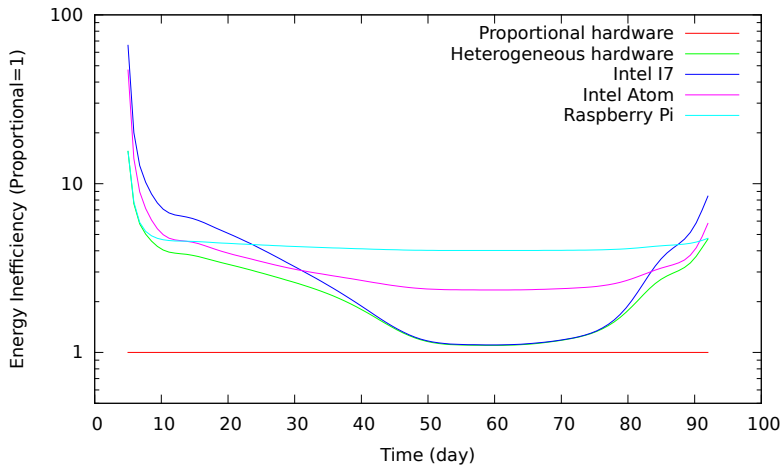




Comparison if using one single data-center

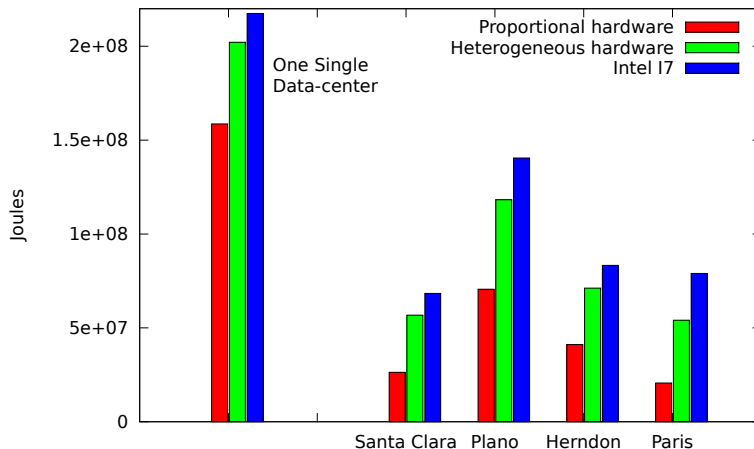


Far reached goal: proportional computing



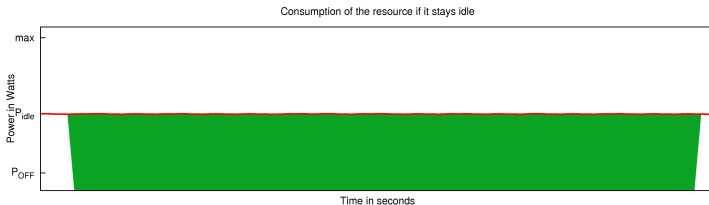
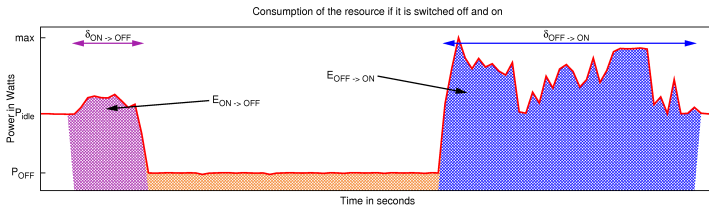


On 98 Football World Cup workload



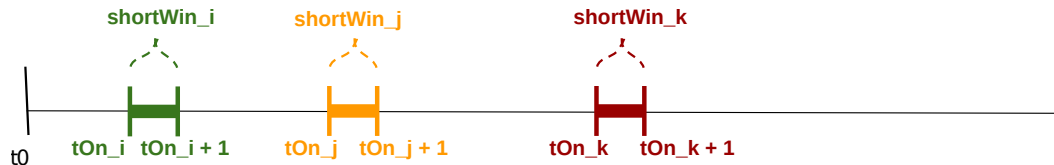
Adding heterogeneity management

- Switching on/off nodes take time
- Switching on/off nodes consumes energy
 - For application reconfiguration
 - For switching on/off the server



Several methods to manage servers

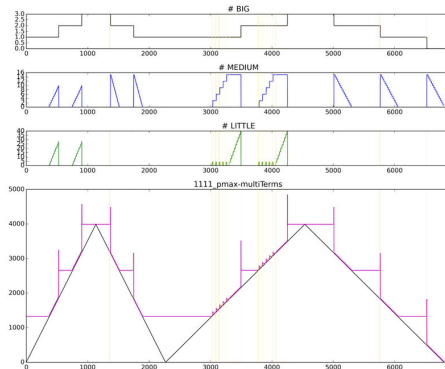
- Exact approach, linear programming
- Heuristics
 - Reactive
 - When overloaded, start new servers, when under-loaded stop some
 - Pro-active
 - Predict the future and decide which server to use



Violaine et al. *Energy Aware Dynamic Provisioning for Heterogeneous Data Centers*, SBAC-PAD 2016

Pro-active Heuristics

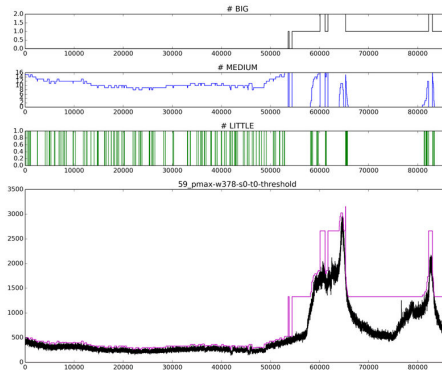
- Predict load and switch-on nodes to guaranty QoS
- When load decrease, switch off servers accordingly
- Stay near the optimal repartition



Violaine et al., *Energy Proportionality in Heterogeneous Data Center Supporting Applications with Variable Load*, ICPDS 2016

Pro-active Heuristics

- Predict load and switch-on nodes to guaranty QoS
- When load decrease, switch off servers accordingly
- Stay near the optimal repartition



Violaine et al., *Energy Proportionality in Heterogeneous Data Center Supporting Applications with Variable Load*, ICPDS 2016

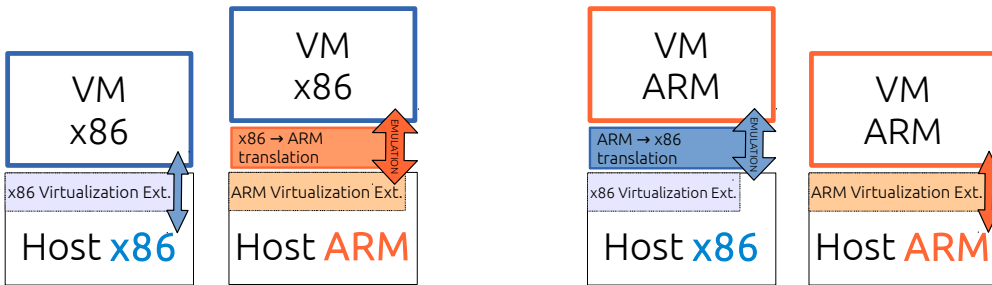


State-full distributed applications

- Classical applications are not maleable
 - Distributed databases
 - OpenMP or MPI applications
- Impossible to change the number of application instance
- Add migration time/energy to Switch on/off

Heterogeneous micro-architectures

- Instance can be migrated between architectures
- Performance depends on architecture of VM and server



- Translation slowdown ratio: 8
- Low load on ARM, high load on x86, x86 VM

Violaine and al. PPL 2005



Plan

- 1 Autonomic loop
- 2 Decision
- 3 Node optimization**
- 4 Current projects and conclusion
- 5 Kernel-level



Toward the future

- A larger number of datacenters
 - Lots of smaller ones (hybrid management)
 - The knowledge: critical resource
 - A large number of diverse sizes
 - Some larger (2016 : 6 300 000 m^2)
- Overall, datacenters will be more integrated in their environment
 - Electrical aspects
 - Thermal aspects



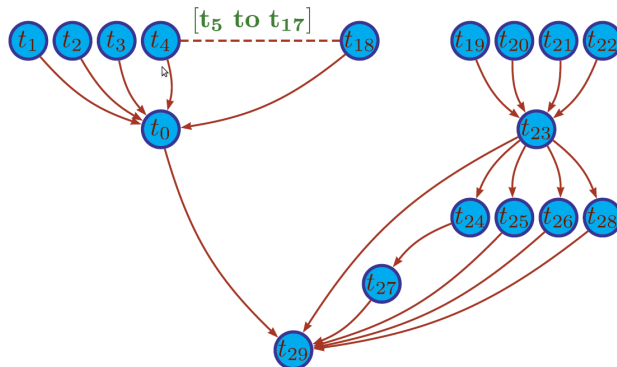
Node optimization

- Three temporalities
 - Large-grained (minute) : Optimal frequency in function of the task graph*
 - 13% of energy savings
 - Medium-grained (second) : Phase detection[†]
 - 20% of energy savings, 3% of time increase
 - Fined-grained (1/10s) : Frequency policy at the kernel level[‡]
 - 25% of energy savings, 1% of time decrease
- No coordination between the three temporality, no objectives

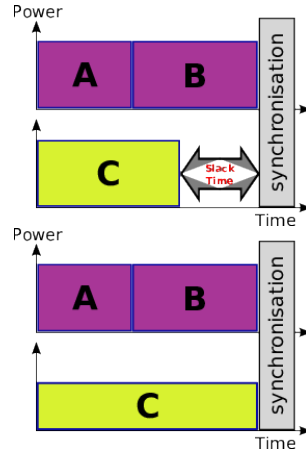
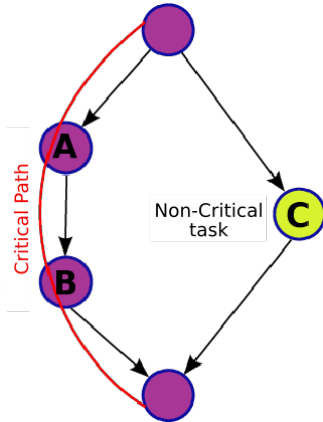
* Tom et al., *Energy-aware simulation with DVFS*, SMPT journal, 2013 [†]Landry et al., *Exploiting performance counters to predict and improve energy performance of HPC systems*, SUSCOM journal, 2014 [‡]Georges et al., *DVFS governor for HPC: Higher, Faster, Greener*, PDP conference, 2015

At the scale of a node: Large-grained

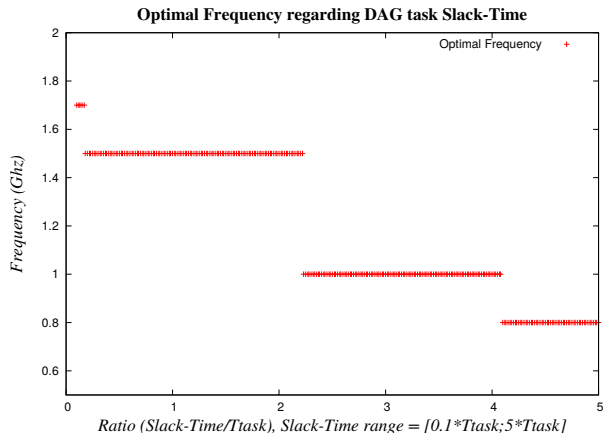
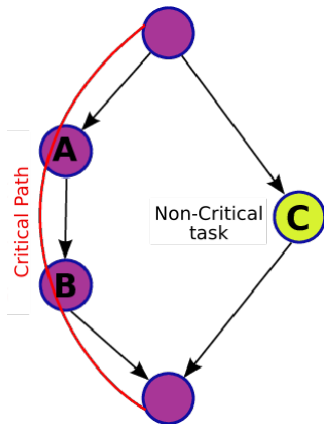
- Use of contextual external information
- Example at the scheduler level: Task DAG



Coordination of node speeds



Coordination of node speeds



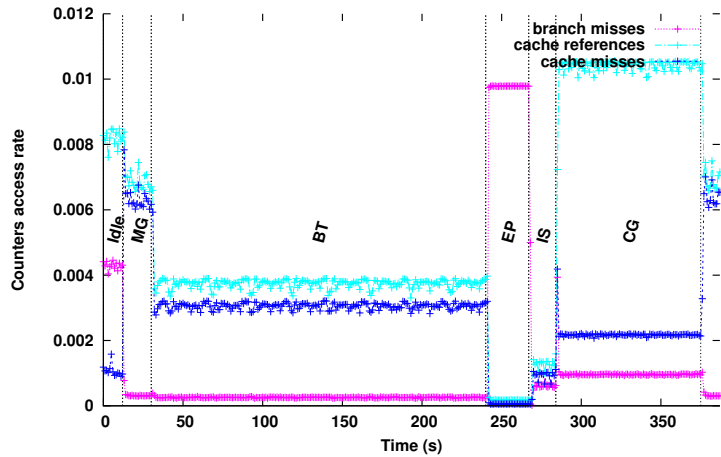


At the level of a node: Medium-grained

- React at medium latency at the level of the node
 - Change the processor frequency
 - Change the hard drive mode
 - Reconfigure the network card
- Detection of the current phase
- React in function of this profile
- Light impact on the infrastructure

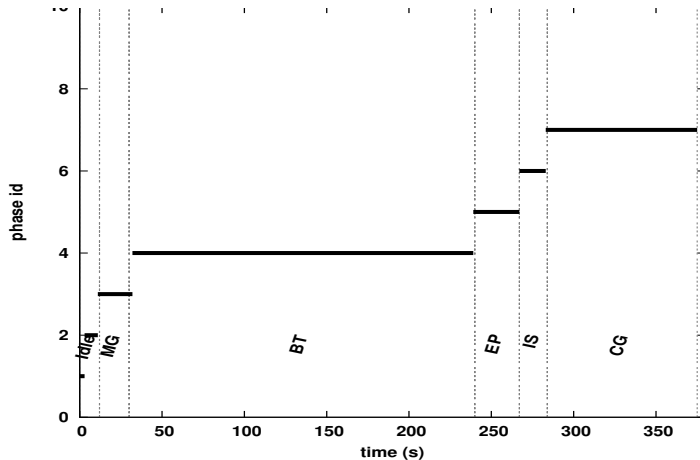


Resource consumption of a complex ap- plication





Phases where resource consumption are constant



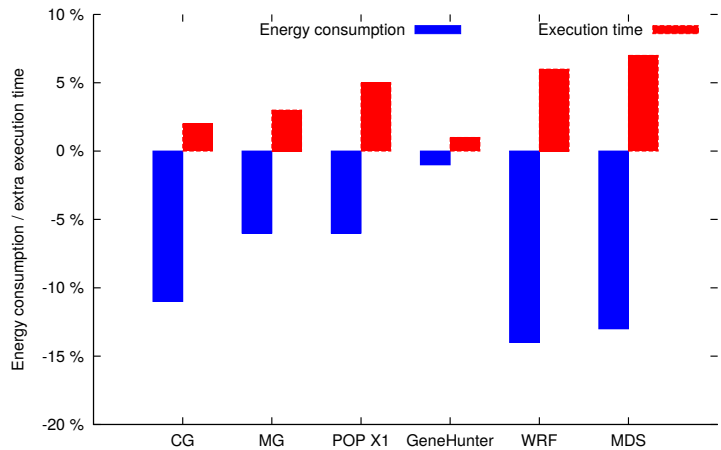


Decision method

Phase label	Possible reconfiguration decisions
compute-intensive	switch off memory banks; send disks to sleep; scale the processor up; put NICs into LPI mode
memory -intensive	scale the processor down; decrease disks or send them to sleep; switch on memory banks
mixed	switch on memory banks; scale the processor up send disks to sleep; put NICs into LPI mode
communication intensive	switch off memory banks; scale the processor down switch on disks
IO-intensive	switch on memory banks; scale the processor down; increase disks, increase disks (if needed)



Energy and performance, 28 nodes





Fine-grained = DVFS ?

Relative values between *performance* and *ondemand* governors

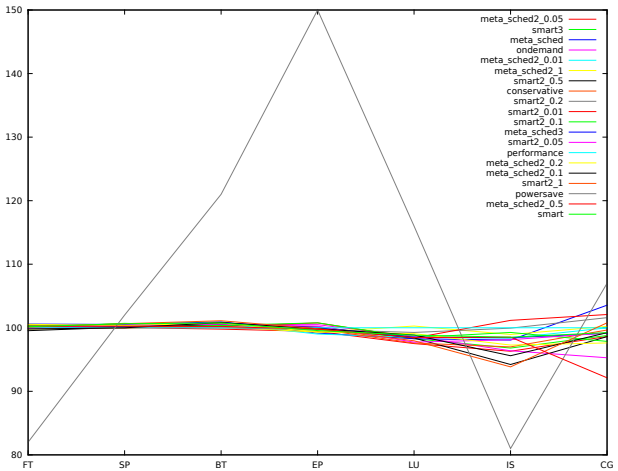
Benchmark	FT	SP	BT	EP	LU	IS	CG
Time increase (%)	0	-3	-1	1	-2	2	0
Energy increase (%)	0	-3	-1	-1	-2	-1	-1

- HPC applications are rarely in Idle... Surprise !
- MPI libraries are spinning

Classical HPC benchmarks from NPB (Nas Parallel Benchmark)

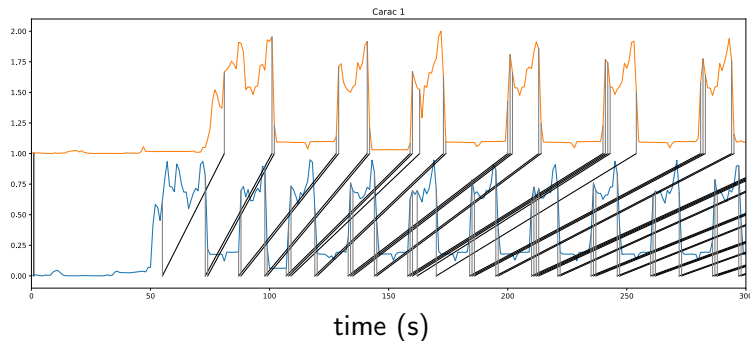


DVFS = function of load



Time malleability by DVFS

- LU application
- Two different frequencies
- Scaled to same time for readability
- Matching of execution phases



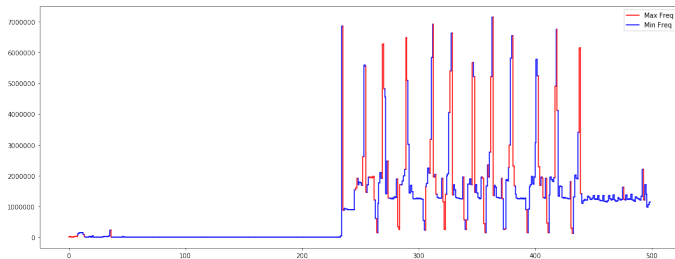


Post-Mortem analysis of best frequency

■ Metric to optimize

■ Example with Energy

Frequency	Time	Energy
Maximum	406s	331J
Minimum	608s	270J
Best	514s	247J

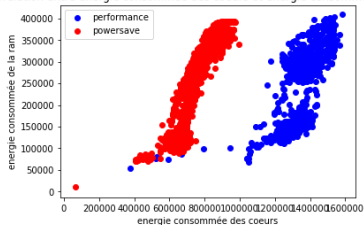




Online detection of best frequency

- Learning using monitored data
 - Hardware performance counters
 - RAPL energy monitor
- 1 hidden layer
- Simple output:
 - 0 → maximum frequency
 - 1 → minimum frequency

correlation entre energie consommée des coeurs et energie consommée de la ram





Yet DVFS has potential

Relative values between *performance* and *powersave* governors

Benchmark	FT	SP	BT	EP	LU	IS	CG
Time increase (%)	36	69	110	159	96	35	83
Energy increase (%)	-18	2	21	50	16	-19	7

- Time rises, but up to 19% or energy consumption reduction !

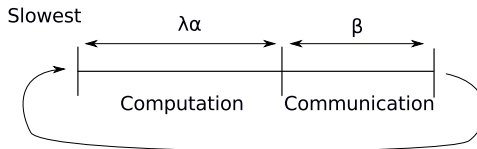
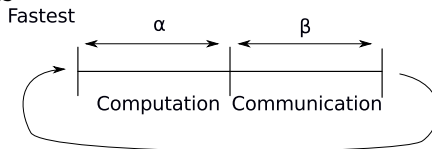


HPC Hypothesis

- State of applications
 - Computing
 - Communications
 - Disk I/O
 - Idle

HPC Hypothesis

- State of applications
 - Computing
 - Communications





Decision

- Energy for maximum frequency

$$(\alpha + \beta)P_1$$

- Energy for minimum frequency

$$(\lambda\alpha + \beta)P_2$$

The processor stays at maximum frequency if it consumes less energy:

$$(\alpha + \beta)P_1 < (\lambda\alpha + \beta)P_2$$



How to measure α and β

- Difficult to measure directly α and β
 - Runtime, not code instrumentation
- Easy to measure network bandwidth (with B_m maximal bandwidth)

$$B_w = B_m \frac{\beta}{\alpha + \beta}$$

- In fact α and β are not important
 - $\frac{\alpha}{\beta}$ is needed, i.e. the ratio between *time to compute* and *time to communicate*



The great mix

Mix and serve:

$$B_w < \frac{B_m}{\lambda - 1} \left(\lambda - \frac{P_1}{P_2} \right) = B_1$$

B_1 : Bandwidth limit at maximum frequency to use or not DVFS

In the opposite direction

$$B_2 = \frac{B_m}{\lambda - 1} \left(\lambda \frac{P_2}{P_1} - 1 \right)$$

B_2 : Bandwidth limit at minimum frequency to use or not DVFS



Network based scheduling

NetSched Algorithm

- Each 10^{th} of a second, do:
 - If Current_Frequency = Slowest frequency and $IBR \leq .9B_1$
 - Change frequency toward Fastest
 - If Current_Frequency = Fastest frequency and $IBR \geq 1.1B_2$
 - Change frequency toward Slowest
 - Else, do nothing

IBR : Incoming Byte Rate



Experimental environment

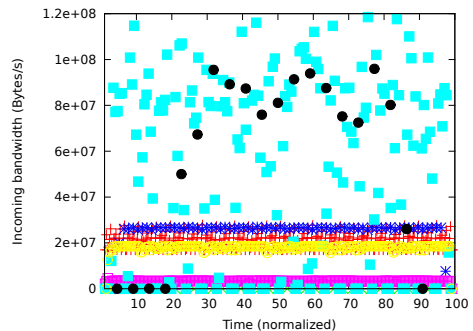
- Servers (thanks Grid5000)
 - Processors : bi Dual-Core AMD Opteron (2218)
 - Memory : 8GB
 - Network card : Gigabyte Ethernet
 - Frequency : 2.6GHz and 1GHz
 - Electrical Power : $P_1 = 280W$ et $P_2 = 152W$
- Benchmark
 - 7 Nas Parallel Benchmark (NPB)
- Governors
 - Performance/Powersave/Ondemand
 - NetSched

$$1.1B_1 \simeq 7.10^7 \text{ et } 0.9B_2 \simeq 3.10^7$$

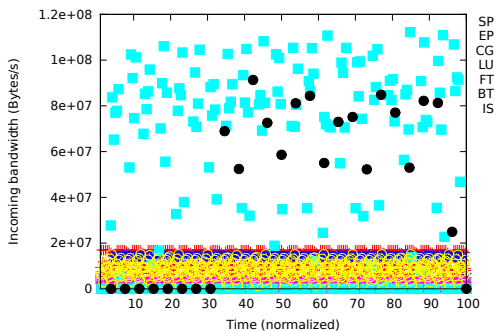


Example of execution

Network communication over time



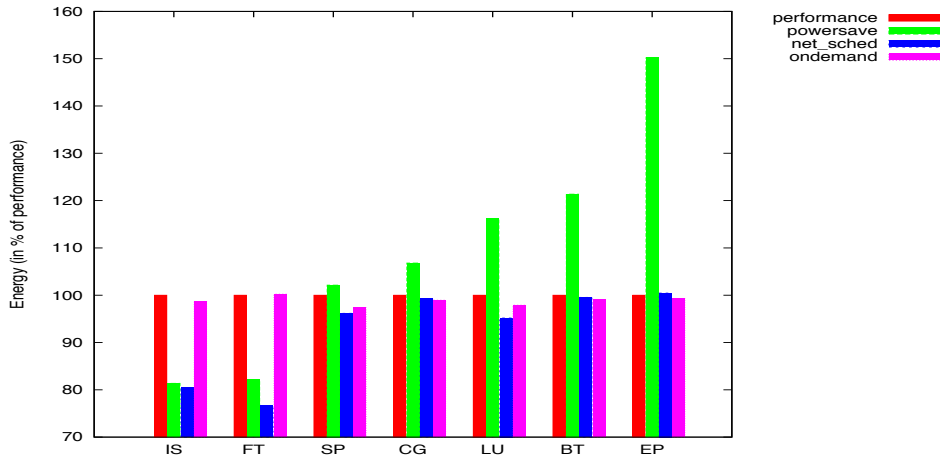
performance



powersave

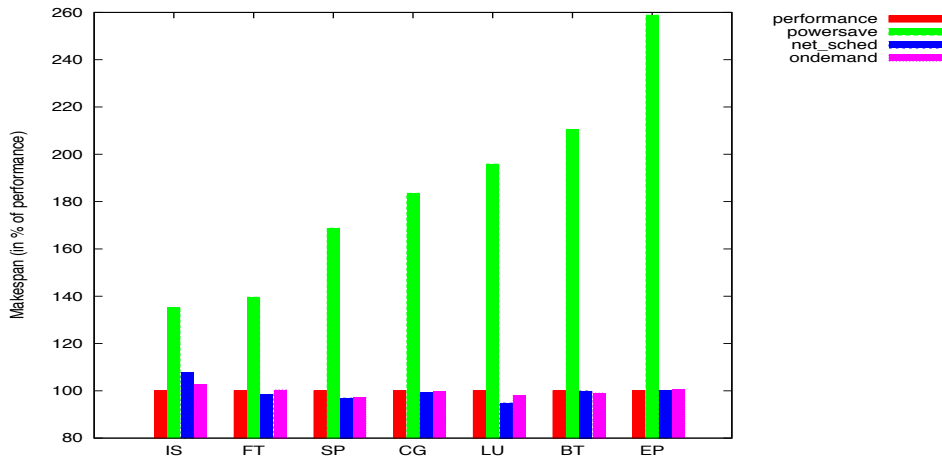


Results: Energy-to-solution





Results: Makespan





The missing link between levels

- An “handmade” work
 - A large number of inter-dependent middlewares
 - Human manipulations
- Toward a decentralized cooperation



Plan

- 1 Autonomic loop
- 2 Decision
- 3 Node optimization
- 4 Current projects and conclusion
- 5 Kernel-level



Plan

- 1 Autonomic loop
- 2 Decision
- 3 Node optimization
- 4 Current projects and conclusion
- 5 Kernel-level**



Hardware is now autonomous

Modern servers:

- Operating System
 - Frequency scaling, deep state
- Processor Firmware
 - Reduce processor frequency

Leads to non deterministic and unwanted behaviors



Thermal management of processors

If temperature over threshold

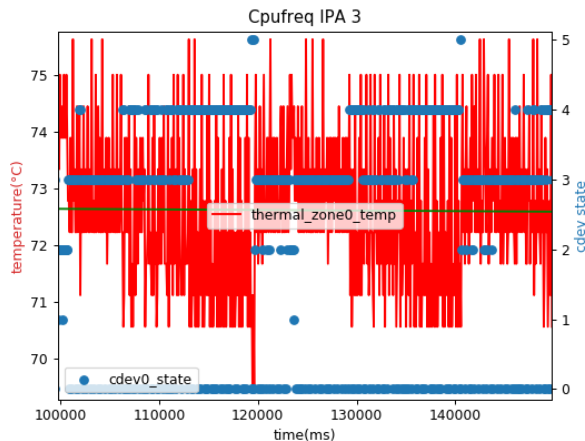
- Reduce frequency

But frequency is discrete

- Low number of possible frequencies
- High cost for chip makers to add frequencies

Current behavior

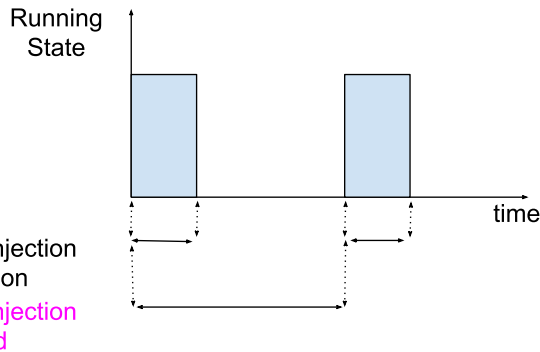
- Blue dots
 - Reduction of frequency level
 - Right y-axis
- Red line
 - Temperature
 - Left y-axis





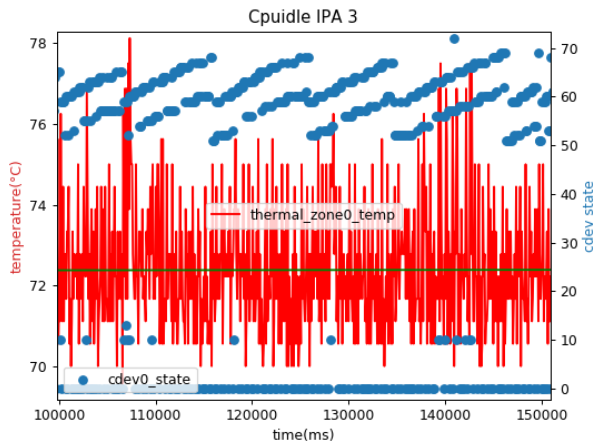
Virtual frequencies

- Idle Injection
 - Each 1/100th of a second
 - Insert deep state sleep
 - Take into account overhead
- Similar to any frequency
- For a target virtual frequency
 - Fix ratio (duration, period)
 - Fix CPU frequency
 - Fix sleep state



With Idle Injection

- Blue dots
 - Percentage of idle
 - Right y-axis
- Red line
 - Temperature
 - Left y-axis
- Max frequency
- Minimum sleep state





Preliminary comparison

- ARM architecture
- Drystone benchmark
 - DMIPS:
performance
- Mean Frequency
when not in Idle
- IPA : Intelligent
Power Allocation
- SWISE: Step Wise

	Mean Temperature	DMIPS	Mean Frequency
IPA DVFS	72.7 °C	46041	1.5GHz
SWISE Freq	74.3 °C	48973	1.6GHz
Idle Injection	72.4 °C	62514	2GHz

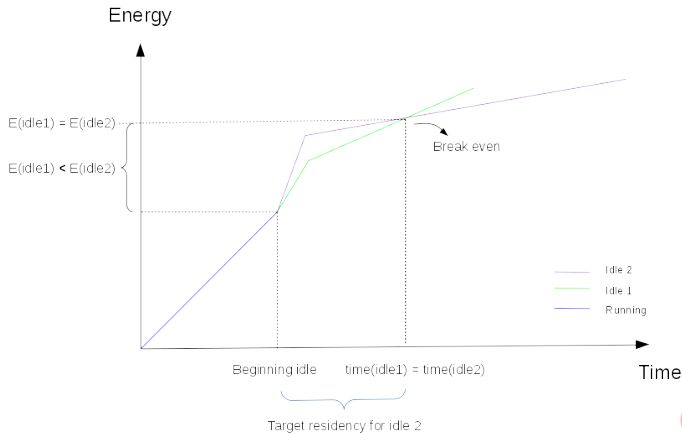


Context of services

- Load
 - HPC workload is simple: 100%
 - Cloud is more complex
- Processors in clouds are often idle
 - Waiting for a timer
 - Waiting for a request
 - ...
- What to do when Idle ?

Break Even

- Several low power mode
- The deeper :
 - The lower the consumption
 - The higher the overhead





Choosing the best level

- In case of error:
 - Too deep
 - Overhead not balanced by savings
 - Too high
 - Consume too much energy
- Information needed:
 - Precise information on each deepstate (latency/power profile)
 - Prediction of next wake-up time



Choosing the best level

- In case of error:
 - Too deep
 - Overhead not balanced by savings
 - Too high
 - Consume too much energy
- Information needed:
 - Precise information on each deepstate (latency/power profile)
 - Prediction of next wake-up time

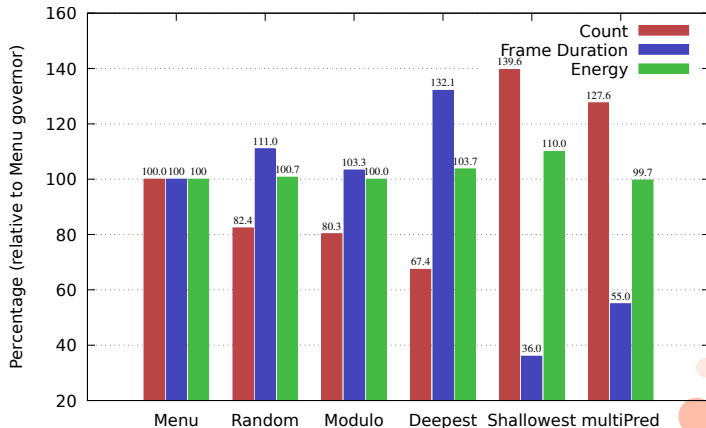
Different sources of wake-up:

- Timer interruption
- I/O devices
- IPI (Inter-processor interrupt)
 - new thread
 - memory interaction
 - ...



Adaptive prediction

- Timers are simple
- I/O and IPI:
 - Circular buffer with last timings
 - Suffix based patterns
- Jankbench
 - Interactive workload
 - Due to heat effect, energy is constant
 - Performance increases
 - Rebound effect

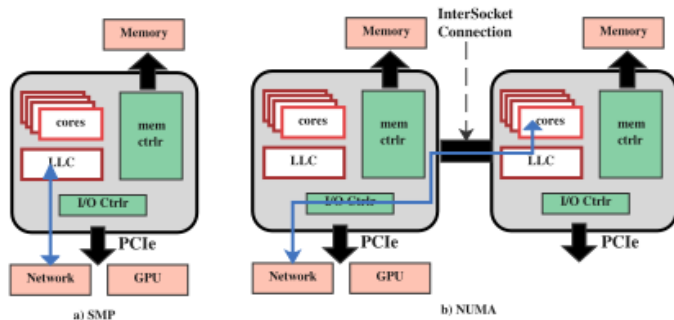




Optimized Resource Allocation on Virtualized Non-Uniform I/O Architectures

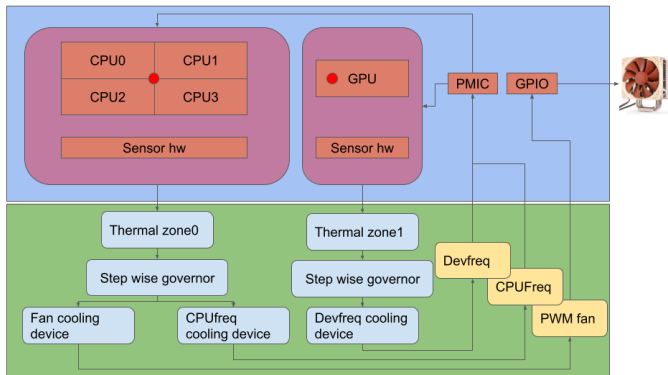
Autonomic loop Decision Node centric Conclusion Kernel-level
Idle-Injection Interrupt prediction Numa Thermal User-Centric Next steps

- Virtualized systems do not have access to topology
- Need of co-scheduling
- Hypervisor NUMA-aware



Thermal Linux Framework

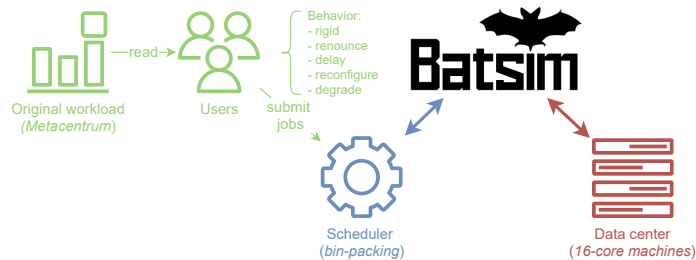
- Balance heat production in dense computer
 - Mobile
 - High-density servers
- Use either DVFS or Idle-injection (virtual DVFS)
- Collaboration with Linaro





Characterization of different user behaviors for demand response in data centers

- Add user model to the system
- Multiple type of user behavior
- Adds new leverages for the scheduler



Open research questions

- Programming paradigms
 - Ability to describe parallelism intuitively
 - Remove the burden from developer
- Runtimes
 - Capability to adapt to particular profiles and their interactions
 - Monitoring & prediction
 - Ability to change kernels in function of context
- Communication between these two levels
- Improvement of RJMS (Resources and Job Management Systems)
 - Spatial management
 - Temporal management

